FIRESIDE CHAT · PREPARED FOR DATA & INSIGHTS · SENIOR DATA ENGINEER

# Nexus, **for Codat.**

**One definition per number. Correct under replay. One truth for people and AI.**

**NEXUS** is the platform I built and ran at Lifepal, an insurance marketplace serving consumers and SMBs: Python on Databricks and Delta, contract-driven and tested end to end, with a dbt semantic layer and an MCP server so people and AI agents query the same certified numbers. Here it is applied to Codat's Insights on SMB data from accounting and ERP systems, bank feeds and commerce platforms: every metric agreed once with its owner, every sync landed replay-safe, every customer's view filtered by consent, and every load checked. Everything below runs at **rfim.github.io/nexus-codat**.

Accounting · ERP | Banking | Commerce | Insights | MCP

BUILT AT lifepal → PRESENTED TO Codat

01 · AGREE **YAML** signed by the owner

Every metric conversation ends in a contract: definition, grain, owner, freshness, access.

02 · LAND **MERGE** replay changes 0 rows

Each sync lands by MERGE on its key, newest version wins; replaying every feed changed nothing.

03 · PROTECT **CONSENT** a row filter per customer

Every product is keyed by customer and company; a revoked consent drops the company everywhere.

04 · PROVE **36/37** checks on every load

Each contract compiles into checks. The one failure is a real £1,250 bank break at C-107.

05 · SERVE **MCP** 4 read-only tools

Agents discover certified metrics and query as the caller, with a citation, or are refused.

PROVEN AT LIFEPAL **6h → 90m** batch · **24h → <5m** latency · **512** assertions, 47 sources · **−60%** ad-hoc requests · **$11k** vs $19k budget

## IF I JOINED YOUR TEAM

the first 90 days

**1 DAYS 1–30**
### Listen and ship
Learn the Insights products, Codat's standard data model and its integrations from their owners. A small fix in production early. Change nothing big.

**2 DAYS 31–60**
### Own a pipeline
Run one Insights pipeline end to end and join its on-call. Write contracts for the rules that live in someone's head.

**3 DAYS 61–90**
### Lay the foundation
First certified metrics in the semantic layer; an MCP prototype over them with an eval set in CI.

**4 DAY 90**
### Measured
Fewer incidents reaching customers, faster diagnosis, and team standards running as code.

**Every day after that:** write production Python, hold the quality bar, and explain the why to engineers and commercial teams alike. Each part of the role is a page you can open at rfim.github.io/nexus-codat.

## DATABRICKS, FEATURE BY FEATURE

| BUILT AT NEXUS · LIFEPAL | APPLIED TO CODAT, IN THIS REPO |
|---|---|
| **Delta MERGE** | Every feed lands by MERGE on its key, newest sync sequence wins. Replaying every feed changed **0 rows**. |
| **canonical model** | Two accounting platforms and an ERP describe one invoice three ways; their contracts map them onto **one canonical invoice**. |
| **SCD2 + as-of** | Chart-of-accounts mappings kept as history and joined at the issue date, never today's mapping. |
| **quarantine** | The quality contract splits bad rows out with the failed rule attached: **4 of 437**. |
| **consent filter** | Products keyed by (customer, company) through active consent; a check fails any row without it. |
| **time travel** | 31 late corrections restated revenue as Delta version 1; **VERSION AS OF 0** still shows what was issued. |
| **semantic layer** | Each Insights metric defined once, with owner, grain, freshness and whether it can be summed. |
| **MCP + evals** | Four read-only tools, run as the caller; **12** eval questions in CI, refusals included. |

## DATA CONTRACT

contracts/products/

The product owner owns the definition; the platform checks it. This YAML compiles into SQL checks, expectations, grants and the consent row filter.

```yaml
product:   insights_cash_flow_monthly
owner:     insights-product-lead@example.com   # signs off
definition: Bank inflows minus outflows, excluding own-account
            transfers and loan drawdowns.
grain:     [customer_id, company_id, currency, month]
reconcile:
  against_sql: SELECT company_id, balance_movement ...
               FROM {silver.bank_balance}
  measure:   net_movement_all
  tolerance: 0.01
access:
  read:          [grp_insights_api, grp_risk_analysts]
  tenant_column: customer_id       # row filter on every read
change: {breaking: owner_approval}  # blast radius on the PR
```

## CI GATE

.github/workflows/ci.yml

Every pull request runs the tests, the contracts and the agent evals before anything merges.

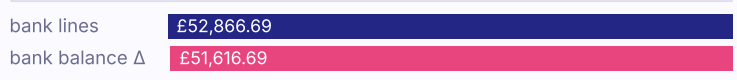
```yaml
on: pull_request
steps:
  - run: pytest -q                  # replay, SCD2, contracts, semantic
  - run: pytest tests/test_mcp.py   # tools, tenant, 12 evals
  - run: ./scripts/build.sh         # Spark run, then the site data
```

## NUMBERS, DONE HONESTLY

worked examples from the run

### Reconciliation — £1,250 at C-107

$$|\ \Sigma \text{ bank lines} - \Delta \text{ bank balance}\ | \le 0.01 \rightarrow \textbf{1 company fails}$$




One payment re-sent under a new id: the key can't catch it, the reconciliation does. The cash-flow check stays red for C-107 until the source is fixed (see the on-call page).

### Restatement — 13 rows restated

$$\Delta = \text{revenue}_{\text{now}} - \text{revenue}_{\text{v0}} = \textbf{−£17,722}$$




August revenue across both customers after 31 backdated credit notes and late voids. The change is −21.3%; the issued version stays queryable for any lender who asks.

### Consent — 8 + 5 companies

$$\text{visible}(c) = \{\ k : (c, k) \in E \wedge \text{revoked}_{c,k} = \varnothing\ \}$$




Two companies are shared by both customers; each sees its own consented view, and agents inherit the same filter.

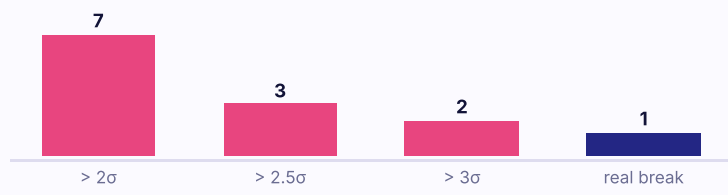
### Commerce — 100.0% in bank

$$\text{match} = \text{payouts found in bank} \le 3 \text{ days} \div \text{payouts} = \textbf{100.0\%}$$

Every commerce payout is looked for in the bank feed: a cross-source check no single integration can give.

### Alerting — hold at 2σ

$$z = (n - \mu_{30}) \div \sigma_{30}$$




On C-107's daily bank volume, a 2σ page would fire 7 times in four months with one real break behind them. Volume goes to a digest; pages are for SLO and reconciliation breaches.

### As-of joins — C-103 · account 4100




✓ as-of join: August keeps its revenue (£1,071.27). × current join would quietly move it to other income.

## THE CODE BEHIND IT

github.com/rfim/nexus-codat

**Delta MERGE · src/nexus/bronze.py**

Bronze is written by MERGE, never INSERT, so a replayed sync page changes nothing.

```python
def upsert_bronze(spark, batch, cfg):
    keys, order, target = cfg["merge_keys"], cfg["ordering_key"], cfg["table"]
    # dedupe inside the batch, so a replayed page can't apply twice
    w = Window.partitionBy(*keys).orderBy(F.col(order).desc())
    latest = batch.withColumn("_rn", F.row_number().over(w)).filter("_rn = 1")
    cond = " AND ".join(f"t.{k} = s.{k}" for k in keys)
    (DeltaTable.forName(spark, target).alias("t")
        .merge(latest.alias("s"), cond)
        .whenMatchedUpdateAll(condition=f"s.{order} > t.{order}")  # newest wins
        .whenNotMatchedInsertAll()
        .execute())
```

**MCP · src/nexus/mcp_server.py**

Agents get governed, read-only tools that run as the caller and cite their source.

```python
@mcp.tool()
def query_metric(name, group_by=None, company_id=None,
                 month_from=None, month_to=None) -> dict:
    """Compile through the semantic layer; run AS THE CALLER."""
    customer = semantic.customer_for(caller())    # consent scope
    sql = semantic.compile_sql(name, group_by or [], customer,
                               company_id, months)
    rows = warehouse.run(sql)
    return {"rows": rows, "sql": sql,
            "citation": {"metric": name, "version": ...}}
# refused by design: uncertified metrics, personal data, writes
```

**Stated plainly.** Lifepal outcomes come from my CV and production work. The Codat side is built from outside, from the role and public information, on synthetic data: the platforms are fictional and no Codat systems or data were used. The numbers in the worked examples are from that synthetic run. Not affiliated with Codat.

R. FIRMAN INSAN MUHAMMAD (VIM) · SENIOR DATA ENGINEER · LONDON · FIRMAN.INSAN.M@GMAIL.COM